

# NautilusTrader 小白指南

开源量化交易框架：Rust 引擎 + Python 策略，回测与实盘同一套代码，纳秒级事件驱动回测

GitHub 28.6K ★

Rust 核心

Python 3.12-3.14

LGPL v3

多资产多交易所

## 疑问即目录

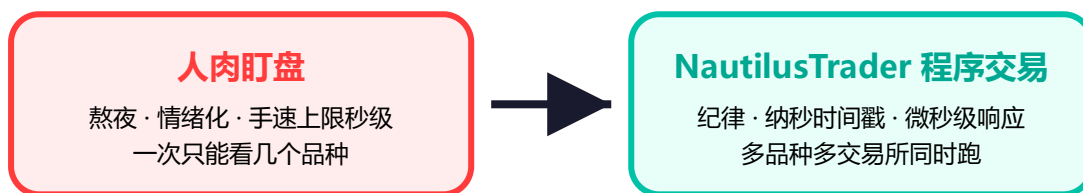
1. 量化交易和 NautilusTrader 是什么？
  2. 为什么底层用 Rust、策略用 Python？
  3. 回测是什么？"回测实盘同一套代码"为什么重要？
  4. 事件驱动架构是怎么回事？
  5. tick、K线 (bar) 、QuoteTick 都是什么？
  6. "纳秒级"时间戳到底意味着什么？
  7. 撮合引擎和模拟交易所 (SIM venue) 是什么？
  8. EMA 均线交叉策略是怎么工作的？
  9. NETTING 和 HEDGING 两种持仓模式有何不同？
  10. 点差、手续费、滑点如何影响回测？
  11. 回测赚钱 = 实盘赚钱吗？
  12. 数据目录 (catalog) 和数据整形器 (wrangler) 是干嘛的？
  13. 和 Backtrader、VectorBT 等前辈框架怎么比？
  14. 想实盘接入真实交易所要做什么？
- 这个项目给我什么 (能力清单)    → 动手实验：跑一次真实回测

## Q1

### 量化交易和 NautilusTrader 是什么？

**量化交易**：把"什么时候买、买多少、什么时候卖"写成**程序规则**，让电脑代替人盯盘下单。人决定策略思想，机器负责纪律和执行速度。

**NautilusTrader**（鸚鵡螺交易者）是澳大利亚 Nautech Systems 开源的**生产级算法交易平台**：它把**历史回测**和**实盘交易**装进同一个事件驱动引擎里，覆盖外汇、股票、期货、期权、加密货币、甚至博彩交易所。



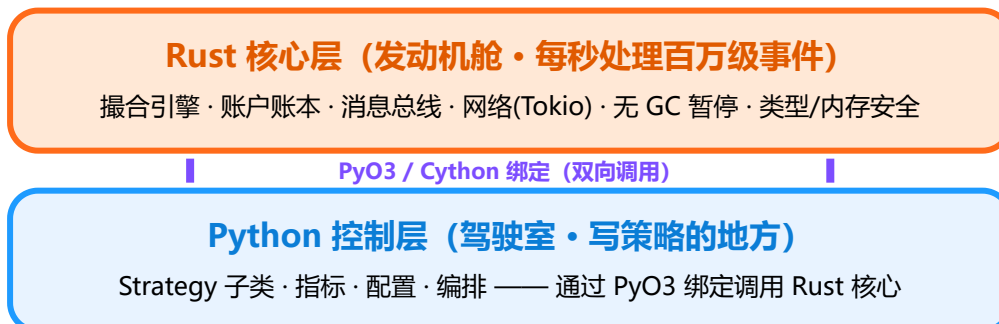
一句话：**用 Rust 造的一台"交易发动机"，Python 当方向盘和仪表盘。**

## Q2

### 为什么底层用 Rust、策略用 Python？

交易软件有两个互相打架的要求：**核心要快到极致且绝不崩溃**（撮合、账本、风控），**策略要改起来快**（今天试均线、明天试机器学习）。

NautilusTrader 的解法是一分为二：



**类比知识库里的"本地推理引擎" (ollama)：模型权重装载、调度这些重活在底层引擎，用户只跟 CLI 打交道。**这里 Rust 负责"绝不能出错、绝不能卡"的部分，Python 负责"天天要改"的部分。整个系统甚至可以全部用 Rust 写，用于对延迟最敏感的场景。

### Q3

## 回测是什么？"回测实盘同一套代码"为什么重要？

**回测 (backtest)** = 拿历史行情数据"重放"一遍，看策略当时会怎么买卖、赚赔多少。是策略上实盘前的 rehearsal (彩排)。

传统量化工具最大的坑：**回测写一套代码、实盘又写一套**——彩排用的剧本和正式演出不是同一份，那彩排还有什么意义？两个系统行为有细微差异（撮合规则、费用、时序），策略一上实盘就变样。

#### 传统做法：两套代码

回测系统 A (简化的撮合)  
实盘系统 B (另一套实现)

行为差异 = 隐形炸弹

#### NautilusTrader: 一套代码

你的 Strategy 类 (不变)

回测引擎 ↔ 实盘节点  
同一份策略，两种运行模式

**同一份策略代码，接上回测引擎就是回测，接上交易所网关就是实盘。这就是推文里"回测与实盘共用同一套代码"的实际含义。**

## Q4

### 事件驱动架构是怎么回事？

NautilusTrader 不是"一次调用算一天"的批处理框架，而是**事件驱动**：行情、订单、成交全是带时间戳的事件，按时间顺序在一条总线上排队处理。



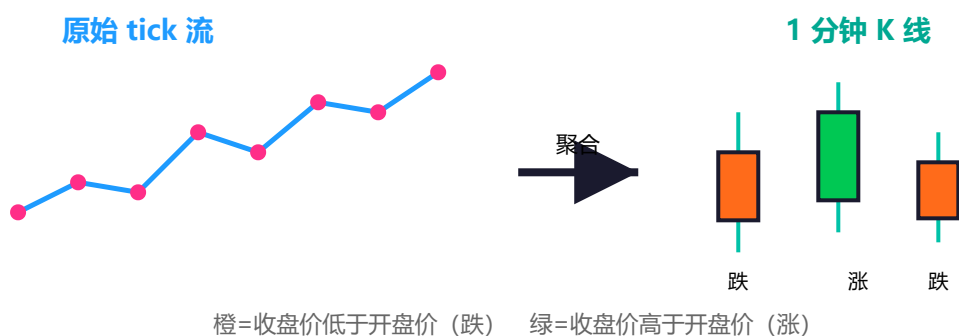
**类比知识库里的"消息队列与异步削峰"：总线就是那个水库，事件就是水流；再类比"Agent 循环"，策略收到事件→做出反应→发出新事件，循环往复。**事件驱动保证了回测和实盘的处理顺序完全一致，这是 Q3"同一套代码"能成立的地基。

## Q5

### tick、K线 (bar) 、QuoteTick 都是什么？

**tick (跳价)**：市场上每一次报价变动的最小记录。外汇里一条 **QuoteTick** 包含：时间戳、买一价 (bid)、卖一价 (ask)、买量、卖量。**买一卖一之间的差价叫点差 (spread)，就是交易成本的一部分。**

**K线 / bar (柱状线)**：把一堆 tick 按时间桶压缩：这一分钟的开盘价、最高价、最低价、收盘价 (OHLC)。K线是 tick 的"摘要"。



实验里我们喂给引擎的是 tick (audusd-ticks.csv 约 10 万条)，策略订阅的是 **1-MINUTE-MID-INTERNAL** K线——**中间的聚合是引擎自动做的** (INTERNAL 聚合器按 (买一+卖一)/2 合成 MID 价 K线)。

## Q6

## "纳秒级"时间戳到底意味着什么？

金融领域“谁先到谁成交”——两个事件相差几微秒，处理顺序不同，结果可能就不同。

NautilusTrader 内部所有事件时间戳都是 **64 位整数纳秒**（不是浮点秒），保证排序精确、不丢精度。

### 实验实测：第一条 tick 的时间戳是

```
first_ts_ns: 1580398089820000000
= 2020-01-30 15:28:09.820000000 UTC (共 10 亿分之一秒精度)
```



**纳秒时间戳是"确定性回测"的前提：同一批数据，重跑一万次，结果一模一样。**

## Q7

## 撮合引擎和模拟交易所 (SIM venue) 是什么?

**撮合引擎 (matching engine)**：真正交易所的心脏，负责把买卖订单按规则配对成交。回测时没有真实交易所，NautilusTrader 在你机器里内置了一个。

**SIM venue (模拟交易所)**：回测里你声明的一个"虚拟交易所"，可以配置：账户类型（现金/保证金）、持仓模式、初始资金、手续费模型、滑点模型。实验里我们建了一个：

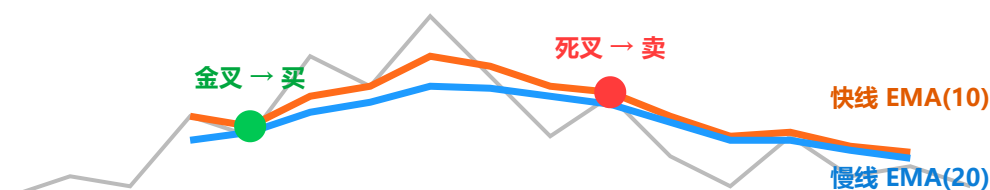
```
engine.add_venue(  
    venue=SIM, # 名字叫 SIM 的模拟交易所  
    oms_type=OmsType.NETTING, # 净额结算：同品种多空自动抵消  
    account_type=AccountType.MARGIN, # 保证金账户（可杠杆）  
    base_currency=USD,  
    starting_balances=[Money(1_000_000, USD)], # 100万美元本金  
    fee_model=MakerTakerFeeModel(...), # 吃单/挂单费率模型  
)
```

策略下的每个市价单，都由这个内置撮合引擎"假装交易所"即时成交并记账——就像学车时的模拟器，仪表全真，但不会真撞车。

## Q8

### EMA 均线交叉策略是怎么工作的？

**EMA (指数移动平均线)**：一种"越近的K线权重越大"的平均价，比简单均线更跟手。两条不同周期的 EMA 画在一起：



灰线 = 原始价格 快线周期短、贴价格更紧；慢线周期长、更平滑

**快线 (周期短) 从下向上穿越慢线 = 金叉 = 买入信号；从上向下穿越 = 死叉 = 卖出信号。**这是量化界的"Hello World"策略：规则简单、人人能懂，但正因为简单，它本身**没有赚钱优势**——官方案例原文就叫它 "a teaching strategy and has no edge"。它的价值在于带你跑通全链路，顺便亲眼看看"交易越频繁，手续费吃掉越多利润"（见实验数据）。

## Q9

### NETTING 和 HEDGING 两种持仓模式有何不同？

同一个品种，先买 10 万又卖 10 万，账户里剩下什么？两种答案：

|      | NETTING (净额结算) | HEDGING (对冲/锁仓)        |
|------|----------------|------------------------|
| 持仓结果 | 两笔抵消，持仓归 0     | 可同时持有 +10 万多单和 -10 万空单 |
| 像什么  | 股票账户：买卖轧差      | 部分外汇平台：锁仓功能            |
| 适合   | 单策略单方向，账本简单    | 多策略共用一个账户、需要锁仓对冲       |

实验用的是 **OmsType.NETTING**：**策略每次金叉先平掉旧仓位再开新仓，引擎自动轧差，账本干净。**官方 FX 教程用 HEDGING，因为外汇保证金实务里多空锁仓很常见。

## Q10

### 点差、手续费、滑点如何影响回测？

回测盈亏 = 行情差价 - 三道真实成本：

$$\begin{array}{c} \text{毛盈亏} \\ \text{行情涨跌赚的} \end{array} - \begin{array}{c} \text{点差} \\ \text{买价卖价差} \end{array} - \begin{array}{c} \text{手续费} \\ \text{每笔固定比例} \end{array} - \begin{array}{c} \text{滑点} \\ \text{成交价偏移} \end{array} \\ = \text{净盈亏 (策略真正的成绩单)}$$

我们的实验把三道成本都打开了：MID 价 K线意味着买卖按中间价成交（理想化），但 `MakerTakerFeeModel` 按合约自带的 0.00002 费率收手续费。实测：10/20 参数一天亏 526.82 美元，其中 227.82 是手续费——成本占亏损的 43%。回测不真实建模成本，等于给自己算命。

## Q11

### 回测赚钱 = 实盘赚钱吗？

不等于！回测好看实盘翻车，是量化新手的第一课。常见死因：

| 陷阱    | 说明                           |
|-------|------------------------------|
| 过拟合   | 参数在历史数据上调到刚好赚钱，换个时间段就失效      |
| 前视偏差  | 代码里不小心用了“未来”的信息（如用了收盘价做开盘决策） |
| 成本低估  | 没算滑点、手续费、点差，信号越频繁死得越惨        |
| 流动性幻觉 | 回测假设单子总能按报价成交，实盘中大单会推动价格     |

**NautilusTrader 的诚实之处：**官方文档专门有一节 "Backtest and live differences"，明说回测无法完全复现实盘中的交易所行为、网络延迟、对账差异。它把回测引擎做得尽量真（真撮合、真账本、真费用），但不向你承诺“回测即实盘”——这正是工程素养的体现。

## Q12

### 数据目录 (catalog) 和数据整形器 (wrangler) 是干嘛的？

量化项目 80% 的时间是洗数据。NautilusTrader 提供两件工具：

**DataCatalog (数据目录)**：把海量行情按 Apache Parquet 列式格式归档，配合 DataFusion 查询引擎用 SQL 快速检索（“给我 2020 年 1 月所有 AUD/USD 的 tick”），大规模回测时按需流式读取，不用全装内存。

**DataWrangler (数据整形器)**：把 pandas DataFrame“翻译”成 Nautilus 强类型对象的转换器。实验里就用了一行：

```
wrangler = QuoteTickDataWrangler(instrument=instrument)
ticks = wrangler.process(df) # DataFrame → list[QuoteTick]（纳秒时间戳、强类型价格
```

用知识库的话说：**wrangler 是“报关口”，DataFrame 是散装货，QuoteTick 是报关后的标准集装箱，引擎只认集装箱。**

## Q13

### 和 Backtrader、VectorBT 等前辈框架怎么比？

|       | Backtrader  | VectorBT       | NautilusTrader   |
|-------|-------------|----------------|------------------|
| 语言核心  | 纯 Python    | 纯 Python/Numba | Rust 核心 + Python |
| 强项    | 简单易学、教学多    | 参数网格扫描极快       | 回测/实盘一套代码、生产级撮合  |
| 定位    | 个人研究玩具      | 研究阶段批量试验       | 从研究一路用到实盘生产      |
| 时间戳精度 | 秒/浮点        | 秒/浮点           | 纳秒整数             |
| 维护状态  | 2017 年后基本停更 | 活跃（PRO 版收费）    | 活跃，双周发版          |

一句话：**Backtrader 像家用缝纫机，VectorBT 像工业裁剪机，NautilusTrader 像服装厂整条产线**——从打样（回测）到量产（实盘）不换设备。

## Q14

### 想实盘接入真实交易所要做什么？

NautilusTrader 用 **模块化适配器 (adapter)** 接入交易所：每个交易所一个适配器，负责该所的行情协议 (WebSocket) 和下单协议 (REST/FIX) 与 Nautilus 内部事件模型之间的双向翻译。



官方已维护币安、Bybit、OKX、dYdX、Interactive Brokers (盈透) 等适配器；任何有 REST/WebSocket/FIX API 的交易所都能照模子写。实盘时策略代码**一行不改**，只是对面从 SIM 换成了真交易所——这就是 Q3 那个承诺的兑现方式。

## 这个项目给我什么 —— 能力清单

### 1. 回测引擎

多品种多策略同时回测，tick/K线/订单簿数据都行，内置撮合、费用、滑点、保证金、隔夜利息模块。

### 2. 实盘执行

同一套策略接真实交易所（币安、盈透等），事件驱动引擎微秒级响应，Redis 可选持久化。

### 3. 高级订单类型

IOC/FOK/GTD、冰山单、OCO/OTO 联动单、post-only/reduce-only 执行指令——真实交易所所有的它都模拟。

### 4. 数据基础设施

Parquet 数据目录 + DataFusion SQL 查询 + 四种数据整形器，大数据量流式回测。

### 5. 200+ 内置指标

EMA/SMA/RSI/MACD/布林带...

`nautilus_trader.indicators` 直接实例化注册。

### 6. AI 训练友好

快得可以直接在平台里训练强化学习交易 Agent（回测即环境）。

结合你的水平，它能干的事：① 把任何“我觉得会赚”的交易想法 1 小时内变成可验证的回测；② 跑参数网格，用数据而不是感觉筛选策略；③ 小资金接加密货币交易所实盘，体验完整量化闭环；④ 学习生产级 Rust/Python 混合架构的范本。

# 动手实验：跑一次真实 AUD/USD 回测（3 步）

## 第 0 步：环境（一次性）

```
pip install -U nautilus-trader pandas
git clone --depth 1 https://github.com/nautechsystems/nautilus_trader repo
```

目的：装稳定版引擎 + 拿到仓库自带的 10 万条真实 AUD/USD tick 数据。

验证：python -c "from nautilus\_trader.backtest.engine import BacktestEngine; print('OK')"

✓ 预期结果：打印 OK（版本 1.231.0，Python 3.12–3.14）

## 第 1 步：跑默认参数回测（快 EMA=10 / 慢 EMA=20）

```
cd exercise
python ema_cross_backtest.py
```

目的：跑通"数据 → 引擎 → 策略 → 报表"全链路。这一步做了四件事：CSV 读成 DataFrame → wrangler 转成 QuoteTick → 引擎用 SIM 交易所撮合 EMA 交叉信号 → 生成盈亏报表。

✓ 预期结果（2026-10-02 实测）：

|                                  |                  |
|----------------------------------|------------------|
| ticks: 99821                     | ← 参与回测的真实 tick 数 |
| first_ts_ns: 1580398089820000000 | ← 纳秒时间戳          |
| signals: 85                      | ← 金叉死叉信号         |
| fills: 170                       | ← 成交笔数（开+平各一次）   |
| realized_pnl: -526.82            | ← 已实现盈亏（美元）      |
| commissions: 227.82              | ← 手续费占了亏损的 43%!  |
| final_balance: 999473.18         | ← 100万本金 → 期末    |

## 第 2 步：改参数，亲眼看见"频率越高死得越快"

```
python ema_cross_backtest.py 5 10    # 更快：5/10
python ema_cross_backtest.py 20 60   # 更慢：20/60
```

目的：同一份数据、同一套代码，只改两个整数，看策略行为的系统性变化。

✓ 预期结果（三组实测对比）：

| EMA 参数       | 信号数 | 成交笔数 | 已实现盈亏(USD) | 手续费(USD) | 期末余额(USD)  |
|--------------|-----|------|------------|----------|------------|
| 5 / 10 (最快)  | 145 | 290  | -883.62    | 388.62   | 999,116.38 |
| 10 / 20 (默认) | 85  | 170  | -526.82    | 227.82   | 999,473.18 |
| 20 / 60 (最慢) | 32  | 64   | -326.76    | 85.76    | 999,673.24 |

**规律：均线越快 → 信号越多 → 手续费越多 → 亏损越大。这就是量化里说的"鞭打 (whipsaw) "：**震荡行情里快线来回穿慢线，每次都白交一次手续费。教学策略没优势是预期的——你从数据中亲手验证了 Q10/Q11 的成本原理，这比读十篇文章都扎实。

**实验源码：** `F:\reminder\nautilus-trader\exercise\ema_cross_backtest.py` (约 170 行, 含中文注释) + `exercise\README.md` (踩坑全记录: pip 稳定版 1.231.0 的 API 与仓库 develop 分支示例差异、CSV 重复时间戳、Money 字符串列等 7 个坑) 。